



BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN CHUYÊN NGÀNH

**HỆ THỐNG KIỂM CHỨNG SỰ THẬT VÀ
PHÂN TÍCH TÍNH TOÀN VỆN CỦA THÔNG
TIN**

Ngành: CÔNG NGHỆ THÔNG TIN

Chuyên ngành: CÔNG NGHỆ THÔNG TIN

Sinh viên thực hiện:

Đinh Khởi Minh (MSSV: 2180605527)

Trần Thanh Phong (MSSV: 2280602382)

Giảng viên hướng dẫn: Nguyễn Hữu Trung

TP. Hồ Chí Minh, 2025

Mục lục

Mục lục	2
Chương 1: TỔNG QUAN	5
1.1. Giới thiệu bối cảnh: Kỷ nguyên của sự hỗn loạn thông tin	5
1.2. Thách thức và vấn đề cấp thiết	5
1.3. Đề xuất giải pháp: Hệ thống FailSafe	6
1.4. Nhiệm vụ đồ án	6
1.4.1 Mục tiêu nghiên cứu	7
1.4.2 Đối tượng nghiên cứu	7
1.4.3 Phạm vi và giới hạn của đồ án	7
1.5. Cấu trúc đồ án	8
Chương 2: CƠ SỞ LÝ THUYẾT	9
2.1. Mô hình ngôn ngữ lớn (Large Language Models - LLMs)	9
2.1.1. Tổng quan về kiến trúc Transformer	9
2.1.2. Cơ chế suy luận: Next-Token Prediction và Chain-of-Thought	9
2.1.3. Hạn chế cốt lõi: Hiện tượng "Ảo giác" (Hallucination)	9
2.2. Kỹ thuật Retrieval-Augmented Generation (RAG)	10
2.2.1. Khái niệm và Vai trò	10
2.2.2. Mô hình Hybrid RAG (Truy xuất lại)	10
2.2.3. Quy trình RAG trong hệ thống FailSafe	11
2.3. Hệ thống Đa tác nhân (Multi-Agent Systems - MAS)	11
2.3.1. Từ Đơn tác nhân đến Đa tác nhân	11
2.3.2. Kiến trúc "Hội đồng thẩm định" (Council of Agents)	11
2.3.3. Cơ chế Đồng thuận (Consensus Mechanism)	12
2.4. Biểu diễn dữ liệu và Xử lý ngôn ngữ tự nhiên (NLP)	12
2.4.1. Vector Embeddings và Tìm kiếm ngữ nghĩa	12
2.4.2. Giải quyết đồng tham chiếu (Coreference Resolution)	13
2.4.3. Đồ thị lập luận cấu trúc (Structured Argumentation Graph - SAG)	13
2.4.4. Phân tích văn phong (Stylometry Analysis)	14
2.5. Kiến trúc Microservices và Xử lý bất đồng bộ	14
2.5.1. Mô hình Microservices	14
2.5.2. Cơ chế Hàng đợi tác vụ (Task Queue) và Xử lý bất đồng bộ	14
Chương 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	16
3.1. Phân tích yêu cầu	16
3.1.1. Yêu cầu chức năng (Functional Requirements)	16
3.1.2. Yêu cầu phi chức năng (Non-functional Requirements)	16
3.2. Kiến trúc hệ thống tổng thể	17
3.2.1. Sơ đồ kiến trúc	17
3.2.2. Mô tả các thành phần chi tiết	17
3.2.3. Cơ chế giao tiếp	18

3.3. Thiết kế luồng xử lý dữ liệu (The Pipeline Design)	19
3.3.1. Sơ đồ luồng dữ liệu tổng quát	19
3.3.2. Chi tiết từng giai đoạn xử lý	20
Giai đoạn 1: Phân rã và Cấu trúc hóa (Decomposition & Structuring)	20
Giai đoạn 2: Kiểm tra Bộ nhớ đệm (Caching Check)	20
Giai đoạn 3: Truy xuất bằng chứng (Retrieval Layer)	20
Giai đoạn 4: Xác thực và Phản biện (Verification Layer - The Council)	21
Giai đoạn 5: Tổng hợp và Báo cáo (Reporting Layer)	21
3.4. Thiết kế Chi tiết các Module chính	21
3.4.1. Module Sàng lọc (Screening Module)	21
3.4.2. Module Truy xuất (Retrieval Module)	21
3.4.3. Module Xác thực (Verification Module)	22
3.5. Thiết kế Cơ sở dữ liệu	22
3.5.1. Cơ sở dữ liệu quan hệ (SQLite)	22
3.5.2. Cơ sở dữ liệu Vector (ChromaDB)	22
3.6. Thiết kế Giao diện người dùng (UI/UX)	23
Chương 4: CÀI ĐẶT VÀ ĐÁNH GIÁ THỰC NGHIỆM	25
4.1. Môi trường cài đặt và Công nghệ sử dụng	25
4.1.1. Môi trường phát triển và triển khai	25
4.1.2. Các thư viện và Framework nòng cốt	25
4.1.3. Cấu hình phần cứng khuyến nghị	26
4.2. Triển khai hệ thống (Deployment)	26
4.2.1. Cấu trúc thư mục dự án	26
4.2.2. Quy trình khởi chạy	26
4.2.3. Kiểm tra trạng thái vận hành	27
4.3. Giao diện người dùng và Chức năng chi tiết	27
4.3.1. Trang Nhập liệu (Input Dashboard)	27
4.3.2. Trang Theo dõi tiến độ (Live Progress Tracking)	28
4.3.3. Trang Kết quả (Result Dashboard)	28
4.4. Kịch bản kiểm thử và Đánh giá (Test Scenarios)	29
4.4.1. Kịch bản 1: Phát hiện Tin giả & Giả khoa học (Pseudoscience)	29
4.4.2. Kịch bản 2: Xác thực Tin tức Khoa học (Factual Verification)	29
4.4.3. Kịch bản 3: Phát hiện Ngụy biện Logic (Logical Fallacy Detection)	30
4.4.4. Đánh giá hiệu năng sơ bộ	30
4.5. Đánh giá và Thảo luận	30
4.5.1. Ưu điểm nổi bật	31
4.5.2. Hạn chế còn tồn tại	31
4.5.3. Kết luận thực nghiệm	31
Chương 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	32
5.1. Kết luận chung	32

5.2. Hướng phát triển trong tương lai	32
TÀI LIỆU THAM KHẢO	33

Chương 1: TỔNG QUAN

1.1. Giới thiệu bối cảnh

Kỷ nguyên của sự hỗn loạn thông tin.

Năm 2022, sự ra đời của ChatGPT và các mô hình ngôn ngữ lớn (LLM) đã đánh dấu một bước ngoặt quan trọng trong lịch sử công nghệ. AI tạo sinh (Generative AI) không chỉ giúp tăng tốc độ xử lý thông tin mà còn thay đổi hoàn toàn cách con người tiếp cận tri thức. Tuy nhiên, đi kèm với sự tiện lợi là một rủi ro chưa từng có: sự bùng nổ của thông tin thiếu kiểm chứng và hiện tượng "ảo giác" của máy móc.

Chúng ta đang sống trong thời đại mà ranh giới giữa sự thật và bịa đặt trở nên mong manh hơn bao giờ hết, khi mà một mô hình AI có thể viết ra một bài báo khoa học giả mạo chỉ trong vài giây, hoặc một tin đồn thất thiệt có thể lan truyền toàn cầu trước khi bất kỳ cơ quan chức năng nào kịp đính chính. Không chỉ là thông tin sai lệch, vấn đề còn nằm ở việc người dùng không còn khả năng phân biệt đâu là nguồn tin đáng tin cậy giữa một biển dữ liệu hỗn loạn.

1.2. Thách thức và vấn đề cấp thiết

Thông qua quá trình nghiên cứu, nhóm nhận diện ba vấn đề cốt lõi dẫn đến nhu cầu cấp thiết:

Thứ nhất: Hiện tượng "Ảo giác thông tin" (AI Hallucination)

Các mô hình LLM hiện tại hoạt động dựa trên cơ chế xác suất (dự đoán từ tiếp theo) chứ không phải dựa trên sự thấu hiểu chân lý. Do đó, chúng thường xuyên tạo ra những thông tin nghe rất trôi chảy, logic nhưng hoàn toàn sai lệch về bản chất.

Ví dụ thực tế: Trong vụ án Mata v. Avianca (2023) tại Mỹ, luật sư Steven Schwartz đã sử dụng ChatGPT để tìm kiếm tiền lệ pháp. Kết quả là ông đã nộp lên tòa án một bản hồ sơ trích dẫn 6 vụ án... hoàn toàn không có thật do AI tự bịa ra. Hậu quả là luật sư này bị phạt tiền và đối mặt với án kỷ luật, cho thấy sự nguy hiểm khi tin tưởng tuyệt đối vào AI mà không có cơ chế kiểm chứng.

Thứ hai: Sự lan truyền của tin giả với tốc độ máy tính

Nếu như trước đây, tin giả do con người tạo ra tốn nhiều thời gian, thì nay AI có thể sản xuất hàng loạt tin giả (fake news), ảnh chế (deepfake) với chi phí gần như bằng không.

Ví dụ thực tế: Vào tháng 5/2023, một hình ảnh do AI tạo ra mô tả vụ nổ tại Lầu Năm Góc (Pentagon) lan truyền trên Twitter. Dù là ảnh giả, nhưng nó đã khiến thị trường chứng khoán Mỹ chao đảo trong chốc lát, chỉ số S&P 500 giảm điểm nhanh chóng trước khi sự thật được phơi bày. Điều này chứng minh rằng con người không còn đủ tốc độ để kiểm chứng thủ công trước "lũ dữ liệu" máy tạo.

Thứ ba: Sự thiếu hụt công cụ kiểm chứng chuyên sâu

Hiện nay đã có các công cụ kiểm tra tin giả (Fact-check tools), tuy nhiên các công cụ hiện tại chủ yếu tập trung vào việc đối chiếu dữ liệu tĩnh hoặc dán nhãn thủ công, chưa có khả năng giải thích logic và tự động suy luận đa bước như con người, không thể hiểu ngữ cảnh sâu và không thể truy xuất ngược nguồn gốc của từng luận điểm.

1.3. Đề xuất giải pháp: Hệ thống FailSafe

Từ những thực trạng trên, nhóm dự án đề xuất xây dựng FailSafe – một hệ thống kiểm chứng thông tin tự động dựa trên kiến trúc AI tạo sinh có trích xuất (RAG) và Hệ đa tác nhân (Multi-agent System).

Giải pháp được thiết kế không chỉ dừng lại ở việc dán nhãn "Đúng/Sai", mà hướng tới việc minh bạch hóa quá trình suy luận. Hệ thống mô phỏng tư duy phản biện của con người thông qua một quy trình xử lý khép kín gồm các tầng bảo vệ:

1. Sàng lọc & Phân rã (Screening & Decomposition): Sử dụng các chỉ số đo lường văn phong (Stylometry) để phát hiện tin giật gân ngay từ đầu, đồng thời bóc tách văn bản phức tạp thành Đồ thị lập luận (SAG) để xử lý từng luận điểm nhỏ
2. Truy xuất lai (Hybrid Retrieval): Kết hợp tìm kiếm ngữ nghĩa (Vector Search) trên cơ sở tri thức nội bộ và tìm kiếm thời gian thực (Web Search) để thu thập bằng chứng đa chiều.
3. Hội đồng thẩm định AI (Council of Agents): Triển khai kiến trúc đa nhân cách, trong đó các AI đóng vai trò chuyên biệt như: Nhà Logic học (Logician) phát hiện ngụy biện, Nhà Nghiên cứu (Researcher) đối chiếu thập bằng chứng, và Nhà Phản biện (Skeptic) bài trừ giả khoa học.

Giải pháp này hướng tới phục vụ các cơ quan quản lý nhà nước, tổ chức báo chí và đơn vị nghiên cứu – những đối tượng đòi hỏi sự chính xác tuyệt đối và khả năng truy xuất nguồn gốc của thông tin.

1.4. Nhiệm vụ đề án

Để giải quyết các vấn đề đã nêu và xây dựng thành công hệ thống FailSafe, đề án tập trung thực hiện các nhiệm vụ chính sau:

1. Nghiên cứu cơ sở lý thuyết: Tìm hiểu về các kiến trúc Mô hình ngôn ngữ lớn (LLMs), kỹ thuật RAG (Retrieval-Augmented Generation) và các phương pháp phát hiện tin giả hiện đại (Stylometry, Claim Decomposition).
2. Xây dựng đường ống xử lý dữ liệu (Data Pipeline): Thiết kế quy trình khép kín từ khâu tiền xử lý (Coreference resolution - giải quyết đồng tham chiếu), phân rã văn bản thành các tuyên bố (Claims extraction) đến xây dựng Đồ thị lập luận (Structured Argumentation Graph - SAG).
3. Phát triển cơ chế kiểm chứng đa tác nhân (Multi-agent Verification): Cài đặt và tinh chỉnh hệ thống "Hội đồng thẩm định" với các vai trò chuyên biệt (Logician, Researcher, Skeptic) để tranh biện và đánh giá bằng chứng.
4. Tích hợp hệ thống truy xuất lai (Hybrid Retrieval): Kết hợp giữa tìm kiếm Vector (trên ChromaDB) và tìm kiếm Web thời gian thực (Google Serper API/Deep Scraping) để đảm bảo nguồn thông tin đối chiếu luôn cập nhật và phong phú.
5. Hiện thực hóa ứng dụng: Xây dựng hệ thống hoàn chỉnh theo kiến trúc Microservices (sử dụng Docker, Flask, Celery, Redis) đảm bảo khả năng mở rộng và xử lý tác vụ không đồng bộ.

1.4.1 Mục tiêu nghiên cứu

Mục tiêu tổng quát:

Xây dựng một hệ thống hỗ trợ kiểm chứng thông tin tự động, có khả năng phát hiện các tuyên bố sai lệch trong văn bản với độ chính xác cao, đồng thời cung cấp giải thích chi tiết cho kết luận đưa ra.

Mục tiêu cụ thể:

- Giảm thiểu ảo giác (Hallucination): Đảm bảo mọi kết luận của AI đều được trích dẫn từ nguồn dữ liệu thực tế (Evidence-based), không để mô hình tự bịa đặt kiến thức.
- Minh bạch hóa quy trình (Explainability): Hệ thống không chỉ trả về kết quả True/False mà phải cung cấp báo cáo lập luận, chỉ rõ tại sao một thông tin bị coi là sai (do ngụy biện logic, sai lệch thời gian hay thiếu bằng chứng).
- Tối ưu hóa tốc độ và hiệu năng: Sử dụng cơ chế Caching (Knowledge Base) để lưu trữ các sự thật đã kiểm chứng, giúp giảm thời gian và chi phí cho các truy vấn lặp lại.

1.4.2 Đối tượng nghiên cứu

- Các mô hình ngôn ngữ lớn (LLMs): Tập trung vào việc kỹ thuật gợi ý (Prompt Engineering) và tinh chỉnh quy trình suy luận của các model như Gemini, GPT-4, Llama.
- Kỹ thuật RAG (Retrieval-Augmented Generation): Các phương pháp truy xuất, xếp hạng (Re-ranking) và tổng hợp thông tin từ dữ liệu bên ngoài.
- Dữ liệu kiểm chứng: Các bài báo, tin tức, tài liệu khoa học và cơ sở dữ liệu đánh giá độ tin cậy nguồn tin (Media Bias/Fact Check - MBFC).
- Kiến trúc hệ thống phân tán: Các kỹ thuật xây dựng hàng đợi tác vụ (Task Queue) và cơ sở dữ liệu Vector.

1.4.3 Phạm vi và giới hạn của đề án

Phạm vi nghiên cứu:

- Dữ liệu đầu vào: Tập trung chính vào xử lý dữ liệu dạng văn bản (Text) tiếng Anh. Hệ thống có module hỗ trợ đa phương thức (hình ảnh, âm thanh) nhưng ở mức độ thử nghiệm chuyển đổi sang văn bản để phân tích.
- Nguồn tri thức: Hệ thống sử dụng nguồn dữ liệu mở trên Internet (Open Web) thông qua các công cụ tìm kiếm và cơ sở tri thức nội bộ (Knowledge Base) được xây dựng từ Wikipedia và các nguồn uy tín.

Giới hạn của đề tài:

- Phụ thuộc vào nguồn tin: Độ chính xác của hệ thống phụ thuộc vào sự tồn tại của thông tin trên Internet. Nếu một sự kiện mới xảy ra chưa được báo chí ghi nhận, hệ thống có thể trả về kết quả "Không đủ bằng chứng" (Inconclusive).
- Độ trễ xử lý: Do áp dụng quy trình phân tích sâu (Deep Analysis) với nhiều bước gọi LLM và cào dữ liệu (Scraping), thời gian xử lý một văn bản dài có thể mất từ vài chục giây đến vài phút, chưa phù hợp cho các ứng dụng yêu cầu thời gian thực (Real-time) tức thì.
- Rào cản ngôn ngữ: Hiện tại hệ thống được tối ưu hóa tốt nhất cho Tiếng Anh (do các bộ dữ liệu MBFC và Prompt mẫu đang sử dụng tiếng Anh). Khả năng xử lý tiếng Việt phụ thuộc vào năng lực của model LLM nền tảng.

1.5. Cấu trúc đồ án

Báo cáo đồ án được trình bày một cách hệ thống qua 5 chương, cụ thể như sau:

Chương 1: Tổng quan

Trình bày bối cảnh ra đời của đề tài, thực trạng vấn đề tin giả và ảo giác của AI trong kỷ nguyên số. Chương này cũng xác định rõ mục tiêu, phạm vi nghiên cứu và giới thiệu sơ lược về giải pháp hệ thống FailSafe nhằm giải quyết các thách thức đã nêu.

Chương 2: Cơ sở lý thuyết

Nghiên cứu các nền tảng công nghệ và thuật toán cốt lõi được sử dụng trong dự án, bao gồm:

- Tổng quan về Mô hình ngôn ngữ lớn (Large Language Models - LLMs) và vấn đề ảo giác (Hallucination).
- Kỹ thuật RAG (Retrieval-Augmented Generation) trong việc bổ sung tri thức cho AI.
- Kiến trúc Đa tác nhân (Multi-agent Systems) trong giải quyết vấn đề phức tạp.
- Cơ sở dữ liệu Vector (Vector Database) và nguyên lý tìm kiếm tương đồng (Semantic Search).
- Các phương pháp phân tích văn phong (Stylometry) và độ tin cậy nguồn tin.

Chương 3: Phân tích và Thiết kế hệ thống

Đây là chương trọng tâm mô tả kiến trúc kỹ thuật của FailSafe:

- Kiến trúc tổng thể: Mô hình Microservices sử dụng Docker, giao tiếp qua Message Queue (Redis/Celery).
- Luồng xử lý dữ liệu (The Pipeline): Chi tiết hóa 5 tầng xử lý: Sàng lọc nhanh (Screening) → Phân rã và xây dựng đồ thị (Decomposition & SAG) → Truy xuất lai (Hybrid Retrieval) → Hội đồng thẩm định (Council of Agents) → Tổng hợp báo cáo.
- Thiết kế dữ liệu: Sơ đồ tổ chức dữ liệu trong SQLite (metadata nguồn tin) và ChromaDB (embeddings kiến thức).

Chương 4: Cài đặt và Đánh giá thực nghiệm

Trình bày quá trình hiện thực hóa hệ thống:

- Môi trường triển khai và các công cụ lập trình (Python, Flask, Docker Compose).
- Mô tả chi tiết các module chức năng chính và giao diện người dùng (Web App).
- Các kịch bản kiểm thử (Test Scenarios) trên các mẫu tin tức thực tế để minh họa khả năng phát hiện tin giả, phân tích lỗi logic và trích xuất bằng chứng của hệ thống.

Chương 5: Kết luận và Hướng phát triển

Tổng kết lại các kết quả đạt được so với mục tiêu ban đầu, phân tích những hạn chế còn tồn tại (về độ trễ, chi phí API, phụ thuộc ngôn ngữ). Từ đó, đề xuất các hướng phát triển trong tương lai như mở rộng hỗ trợ đa ngôn ngữ, xử lý đa phương thức (hình ảnh/video) và tối ưu hóa thời gian phản hồi thời gian thực.

Chương 2: CƠ SỞ LÝ THUYẾT

2.1. Mô hình ngôn ngữ lớn (Large Language Models - LLMs)

2.1.1. Tổng quan về kiến trúc Transformer

Mô hình ngôn ngữ lớn (LLM) là các mạng nơ-ron nhân tạo có số lượng tham số khổng lồ (từ hàng tỷ đến hàng nghìn tỷ tham số), được huấn luyện trên lượng dữ liệu văn bản không gán nhãn quy mô lớn để thực hiện các tác vụ hiểu và sinh ngữ nghĩa.

Nền tảng của các LLM hiện đại được sử dụng trong đồ án này (như Google Gemini, OpenAI GPT) là kiến trúc Transformer, được giới thiệu bởi Vaswani và cộng sự. Điểm đột phá của Transformer nằm ở cơ chế Sự chú ý bản thân (Self-Attention).

- Cơ chế hoạt động: Self-Attention cho phép mô hình đánh trọng số (weight) tầm quan trọng của từng từ trong mối tương quan với tất cả các từ khác trong cùng một ngữ cảnh đầu vào, bất kể khoảng cách vị trí.
- Ý nghĩa với đồ án: Nhờ cơ chế này, hệ thống có khả năng xử lý các đoạn văn bản dài và hiểu được các mối quan hệ ngữ nghĩa phức tạp, chẳng hạn như việc đối chiếu một "Tuyên bố" (Claim) với một đoạn "Bằng chứng" (Evidence) nằm cách xa nhau trong văn bản gốc.

2.1.2. Cơ chế suy luận: Next-Token Prediction và Chain-of-Thought

Về bản chất, LLM hoạt động dựa trên nguyên lý xác suất thống kê: dự đoán từ (hoặc token) tiếp theo có khả năng xuất hiện cao nhất dựa trên chuỗi các từ trước đó (Next-Token Prediction).

Tuy nhiên, để thực hiện các tác vụ kiểm chứng thông tin đòi hỏi tư duy logic, đồ án không chỉ dựa vào khả năng dự đoán từ đơn thuần mà áp dụng kỹ thuật Chain-of-Thought (CoT) Prompting.

- Khái niệm: Thay vì yêu cầu mô hình đưa ra câu trả lời ngay lập tức, CoT hướng dẫn mô hình phân rã vấn đề thành các bước suy luận trung gian.
- Ứng dụng thực tế: Trong module ClaimVerify của hệ thống FailSafe, thay vì hỏi "Câu này đúng hay sai?", hệ thống yêu cầu LLM thực hiện 3 bước:
 - (1) Đánh giá độ tin cậy nguồn tin.
 - (2) Tóm tắt nội dung bằng chứng.
 - (3) So sánh để đưa ra kết luận.

Kỹ thuật này giúp cải thiện đáng kể khả năng suy luận của mô hình đối với các vấn đề phức tạp.

2.1.3. Hạn chế cốt lõi: Hiện tượng "Ảo giác" (Hallucination)

Đây là thách thức lớn nhất của LLM và là động lực chính để phát triển hệ thống FailSafe. Do được huấn luyện để tối ưu hóa sự trôi chảy của ngôn ngữ (fluency) thay vì độ chính xác của sự thật (factuality), LLM thường gặp phải hiện tượng Ảo giác (AI Hallucination).

Ảo giác được định nghĩa là khi mô hình tạo ra nội dung sai lệch, vô lý hoặc không trung thực so với dữ liệu nguồn, nhưng lại được trình bày với văn phong rất tự tin và thuyết phục. Có hai dạng ảo giác chính ảnh hưởng đến bài toán kiểm chứng:

1. Ảo giác ngoại sinh (Extrinsic Hallucination): Mô hình bịa đặt ra các thông tin không có trong dữ liệu huấn luyện (ví dụ: bịa ra một vụ án không có thật).

2. Ảo giác nội sinh (Intrinsic Hallucination): Mô hình xử lý sai thông tin được cung cấp, dẫn đến kết luận mâu thuẫn với dữ liệu đầu vào (ví dụ: đọc tài liệu A nhưng lại tóm tắt sai ý của tài liệu đó).

Do đó, việc sử dụng LLM đơn lẻ để kiểm chứng thông tin là không an toàn. Đồ án giải quyết vấn đề này bằng cách kết hợp LLM với các nguồn tri thức bên ngoài thông qua kỹ thuật RAG (sẽ được trình bày ở phần 2.2).

2.2. Kỹ thuật Retrieval-Augmented Generation (RAG)

2.2.1. Khái niệm và Vai trò

Retrieval-Augmented Generation (RAG) là một kỹ thuật tiên tiến trong lĩnh vực AI, được thiết kế để "cấp quyền" cho các Mô hình ngôn ngữ lớn (LLM) truy cập vào dữ liệu bên ngoài thời gian thực.

Nếu coi LLM là một "bộ não" có khả năng tư duy nhưng trí nhớ bị đóng băng tại thời điểm huấn luyện (Knowledge Cutoff), thì RAG chính là "cuốn từ điển" hoặc "công cụ tìm kiếm" mà bộ não này sử dụng để tra cứu thông tin mới nhất.

Trong hệ thống FailSafe, RAG đóng vai trò cốt lõi để đảm bảo tính xác thực (Factuality). Thay vì để LLM tự bịa ra câu trả lời (dễ dẫn đến ảo giác), hệ thống bắt buộc mô hình phải trả lời dựa trên các bằng chứng cụ thể vừa tìm được.

2.2.2. Mô hình Hybrid RAG (Truy xuất lại)

Đồ án áp dụng kiến trúc Hybrid RAG, kết hợp sức mạnh của hai phương pháp truy xuất dữ liệu khác nhau để tối ưu hóa độ chính xác và độ phủ thông tin.

a. Truy xuất dựa trên Vector (Dense/Vector Retrieval)

- Cơ chế: Sử dụng mô hình Sentence Transformers (như intfloat/e5-base hoặc all-MiniLM-L6-v2) để chuyển đổi truy vấn (Query) và tài liệu (Document) thành các vector trong không gian nhiều chiều. Sau đó, thuật toán tìm kiếm láng giềng gần nhất (ANN - Approximate Nearest Neighbor) được sử dụng để tìm các đoạn văn bản có ý nghĩa tương đồng nhất.
- Ứng dụng trong đồ án: Được sử dụng trong module Knowledge Base (chạy trên ChromaDB). Khi một tuyên bố (Claim) mới xuất hiện, hệ thống sẽ tìm trong cơ sở dữ liệu xem tuyên bố này (hoặc các tuyên bố tương tự về mặt ngữ nghĩa) đã từng được kiểm chứng chưa. Việc này giúp hệ thống "học" từ quá khứ và phản hồi nhanh chóng mà không cần tốn chi phí tìm kiếm lại.

b. Truy xuất dựa trên Web (Web-based Retrieval)

- Cơ chế: Khi thông tin không có trong cơ sở dữ liệu nội bộ (Vector DB), hệ thống kích hoạt module tìm kiếm Web. Module này sử dụng các API tìm kiếm (như Google Serper) để quét Internet theo thời gian thực.
- Quy trình Deep Scraping: Không chỉ dừng lại ở việc đọc tiêu đề hay đoạn tóm tắt (snippet) của Google, hệ thống tích hợp trình cào dữ liệu sâu (DeepScraper). Nó tự động truy cập vào các đường dẫn (URL) được đánh giá là uy tín, tải về toàn bộ nội dung bài viết, lọc bỏ quảng cáo/nhiều HTML để lấy ra văn bản thuần túy làm bằng chứng.

2.2.3. Quy trình RAG trong hệ thống FailSafe

Quy trình hoạt động của RAG trong đồ án tuân theo chu trình chuẩn: Retrieve - Read - Generate (Truy xuất - Đọc hiểu - Tạo sinh), cụ thể hóa qua các bước:

1. Query Generation: Từ một tuyên bố cần kiểm chứng, module QueryGenerator sử dụng LLM để tạo ra nhiều từ khóa/câu hỏi tìm kiếm tối ưu.
2. Retrieval: Hệ thống thực hiện tìm kiếm song song trên cả Vector DB và Web Search.
3. Ranking & Filtering: Các kết quả trả về được đánh giá sơ bộ. Chỉ những nguồn tin có độ tin cậy (Trust Level) cao và nội dung liên quan mật thiết mới được giữ lại.
4. Augmentation: Các bằng chứng đã lọc được đưa vào Prompt (câu lệnh) cùng với tuyên bố gốc, gửi đến LLM để thực hiện bước xác thực cuối cùng.

2.3. Hệ thống Đa tác nhân (Multi-Agent Systems - MAS)

2.3.1. Từ Đơn tác nhân đến Đa tác nhân

Trong các hệ thống AI truyền thống (Single-Agent), một mô hình ngôn ngữ duy nhất chịu trách nhiệm cho toàn bộ quy trình xử lý. Cách tiếp cận này thường gặp phải vấn đề về "Sự trôi dạt mục tiêu" (Goal Drifting) – khi mô hình quên mất các ràng buộc ban đầu trong quá trình suy luận dài, hoặc "Thiên kiến mô hình" (Model Bias) – khi mô hình có xu hướng thiên vị một chiều.

Hệ thống Đa tác nhân (MAS) giải quyết vấn đề này bằng cách chia nhỏ một tác vụ phức tạp cho nhiều thực thể AI (Agents) riêng biệt. Mỗi tác nhân được định nghĩa với một vai trò, mục tiêu và cá tính riêng, hoạt động độc lập hoặc phối hợp để đạt được kết quả chung tối ưu hơn.

2.3.2. Kiến trúc "Hội đồng thẩm định" (Council of Agents)

Lấy cảm hứng từ quy trình phản biện khoa học (Peer Review), đồ án thiết kế module xác thực thông tin (ClaimVerify) dưới dạng một hội đồng gồm 3 tác nhân chuyên biệt. Đây là một ứng dụng của kỹ thuật Role-Playing Prompting, giúp định hướng suy luận của LLM đi vào chiều sâu.

a. The Logician (Nhà Logic học)

- Vai trò: Tập trung phân tích cấu trúc lập luận và tính nhất quán nội tại của thông tin.
- Nhiệm vụ: Phát hiện các lỗi ngụy biện logic (như Ad Hominem - tấn công cá nhân, Straw Man - ngụy biện rơm), các mâu thuẫn nhân quả, và đặc biệt là các lỗi về niên đại (Anachronisms) – ví dụ: một văn bản cổ đại nhưng lại sử dụng đơn vị đo lường hiện đại (mét, giây).
- Mục tiêu: Đảm bảo tính hợp lý (Validity) của tuyên bố.

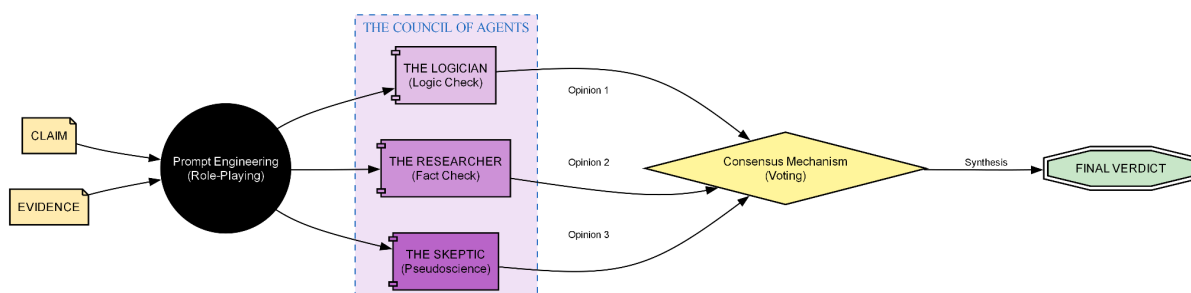
b. The Researcher (Nhà Nghiên cứu)

- Vai trò: Đại diện cho tư duy thực chứng, dựa trên dữ liệu và đồng thuận khoa học.
- Nhiệm vụ: Đánh giá bằng chứng dựa trên Tháp bằng chứng (Hierarchy of Proof). Tác nhân này ưu tiên các dữ liệu từ thực nghiệm, báo cáo khoa học (Tier 1) hơn là các giả thuyết, giai thoại hay ý kiến cá nhân (Tier 3).

- Mục tiêu: Đảm bảo tính xác thực (Factuality) dựa trên bằng chứng vật lý hoặc tài liệu uy tín.

c. The Skeptic (Nhà Phản biện)

- Vai trò: Đại diện cho tư duy hoài nghi khoa học, đóng vai trò là "người gác cổng" chống lại giả khoa học (Pseudoscience).
- Nhiệm vụ: Áp dụng nguyên tắc Dao cạo Occam (Occam's Razor): "Khi có nhiều giả thuyết cùng giải thích một hiện tượng, giả thuyết đơn giản nhất (ít giả định nhất) thường là đúng". Tác nhân này sẽ bác bỏ các tuyên bố phi thường nếu không có bằng chứng phi thường đi kèm.
- Mục tiêu: Loại bỏ các thuyết âm mưu và thông tin gây nhiễu (False Balance).



Ảnh 2.1 Sơ đồ Kiến trúc Hội đồng thẩm định

2.3.3. Cơ chế Đồng thuận (Consensus Mechanism)

Sau khi mỗi tác nhân đưa ra đánh giá độc lập (Gán nhãn: SUPPORTS, REFUTES, hoặc IRRELEVANT kèm theo lý luận), hệ thống thực hiện cơ chế tổng hợp kết quả:

- Nếu các tác nhân đồng thuận: Kết luận được đưa ra với độ tin cậy cao.
- Nếu có sự bất đồng: Hệ thống sẽ ưu tiên ý kiến của tác nhân có trọng số cao hơn tùy theo ngữ cảnh (ví dụ: trong các vấn đề khoa học, ý kiến của The Researcher và The Skeptic sẽ có trọng lượng hơn The Logician nếu bằng chứng vật lý rõ ràng).

Cơ chế này giúp hệ thống FailSafe mô phỏng được quy trình tư duy phản biện đa chiều của con người, giảm thiểu rủi ro sai sót so với việc chỉ tin tưởng vào một mô hình AI đơn lẻ.

2.4. Biểu diễn dữ liệu và Xử lý ngôn ngữ tự nhiên (NLP)

2.4.1. Vector Embeddings và Tìm kiếm ngữ nghĩa

Trong các hệ thống tìm kiếm truyền thống (Lexical Search), máy tính chỉ so sánh sự trùng khớp của từ khóa (keyword matching). Phương pháp này bộc lộ hạn chế lớn khi người dùng sử dụng các từ đồng nghĩa hoặc diễn đạt cùng một ý theo cách khác nhau. Đồ án giải quyết vấn đề này bằng kỹ thuật Vector Embeddings.

- Nguyên lý: Sử dụng các mô hình học sâu (như Sentence-BERT trong thư viện sentence-transformers) để chuyển đổi văn bản (câu, đoạn văn) thành các vector số học trong không gian nhiều chiều (thường là 384 hoặc 768 chiều).
- Tính chất: Trong không gian này, các văn bản có nội dung ngữ nghĩa tương đồng sẽ nằm gần nhau về mặt hình học, bất kể chúng dùng từ ngữ khác nhau.

- Ứng dụng (Semantic Search): Hệ thống sử dụng thuật toán Cosine Similarity (Độ tương đồng Cosine) để đo góc giữa các vector. Điều này cho phép module KnowledgeBase tìm ra các sự thật đã được kiểm chứng tương ứng với một tuyên bố mới, ngay cả khi cách diễn đạt thay đổi hoàn toàn.

2.4.2. Giải quyết đồng tham chiếu (Coreference Resolution)

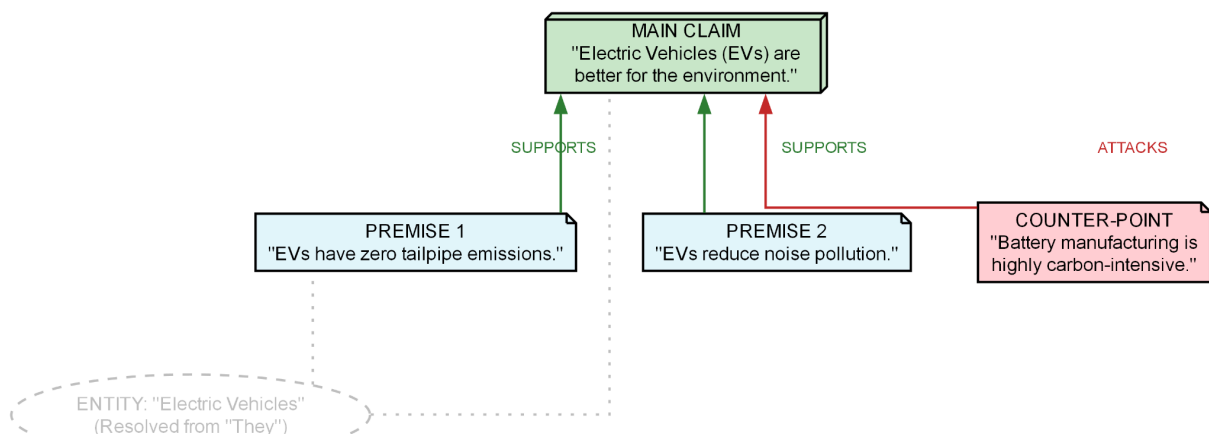
Một thách thức lớn khi phân tích văn bản tự động là sự nhập nhằng của các đại từ nhân xưng (he, she, it, they...) hoặc danh từ chỉ định (this, that...).

- Ví dụ: Trong câu "Elon Musk thành lập SpaceX. Ông ấy là CEO.", máy tính cần hiểu "Ông ấy" chính là "Elon Musk". Nếu tách câu thứ hai ra độc lập ("Ông ấy là CEO"), thông tin sẽ trở nên vô nghĩa.

Đồ án tích hợp module FastCoref – một mô hình chuyên dụng cho tác vụ giải quyết đồng tham chiếu.

- Quy trình: Trước khi phân tích, toàn bộ văn bản đầu vào đi qua module này. Mô hình sẽ xác định các cụm từ cùng chỉ một thực thể (Clusters) và thay thế các đại từ bằng tên thực thể chính (Head Mention).
- Kết quả: Văn bản sau xử lý trở nên tường minh (Explicit), đảm bảo rằng khi hệ thống phân rã văn bản thành các câu đơn (Atomic Claims), mỗi câu đều chứa đầy đủ thông tin ngữ cảnh để có thể kiểm chứng độc lập.

2.4.3. Đồ thị lập luận cấu trúc (Structured Argumentation Graph - SAG)



Ảnh 2.2 Sơ đồ Đồ thị lập luận cấu trúc

Văn bản không chỉ là tập hợp các câu rời rạc mà là một cấu trúc logic chặt chẽ. Để máy tính "hiểu" được mạch văn, đồ án mô hình hóa dữ liệu dưới dạng Đồ thị tri thức (Knowledge Graph), cụ thể là SAG (Structured Argumentation Graph).

- Cấu trúc đồ thị: Dữ liệu được biểu diễn dưới dạng các bộ ba (Triples) theo chuẩn JSON-LD:
 1. Nút (Nodes): Đại diện cho các Tuyên bố (Claims) hoặc Thực thể (Entities).
 2. Cạnh (Edges): Đại diện cho mối quan hệ lập luận, ví dụ: SUPPORTS (Hỗ trợ), ATTACKS (Phản bác), EXPLAINS (Giải thích).
- Lợi ích: Việc chuyển đổi văn bản thành đồ thị giúp hệ thống:
 3. Nhận diện được luận điểm chính (Main Claim) và các luận cứ phụ trợ (Premises).

4. Truy vết ngược (Traceability): Khi một bằng chứng bác bỏ luận cứ phụ, hệ thống có thể suy luận sự ảnh hưởng của nó lên luận điểm chính thông qua các cạnh kết nối.
5. Trực quan hóa tư duy: Giúp người dùng nhìn thấy bức tranh tổng thể về logic của bài viết trên giao diện người dùng.

2.4.4. Phân tích văn phong (Stylometry Analysis)

Bên cạnh nội dung, "cách viết" cũng chứa đựng nhiều tín hiệu về độ tin cậy. Đồ án áp dụng kỹ thuật Stylometry (Đo lường phong cách) để sàng lọc tin giả ngay từ bước đầu.

- Các chỉ số đo lường:
 - + Entropy thông tin: Đo lường độ phong phú và tính ngẫu nhiên của từ vựng. Văn bản tin giả thường có entropy thấp (từ ngữ lặp lại, nghèo nàn).
 - + Cảm xúc (Sentiment) & Giật gân (Sensationalism): Đếm tần suất xuất hiện của các từ ngữ mang tính kích động cảm xúc mạnh (shocking, exposed, secret...).
- Mô hình thống kê: So sánh các chỉ số của văn bản đầu vào với bộ dữ liệu chuẩn (Corpus) tin tức chính thống để phát hiện các bất thường (Anomaly Detection).

2.5. Kiến trúc Microservices và Xử lý bất đồng bộ

2.5.1. Mô hình Microservices

Để xây dựng một hệ thống phức tạp, có khả năng mở rộng và bảo trì dễ dàng, đồ án không sử dụng kiến trúc nguyên khối (Monolithic) mà áp dụng mô hình Microservices (Kiến trúc vi dịch vụ). Hệ thống được chia tách thành các dịch vụ nhỏ, độc lập, mỗi dịch vụ đảm nhận một chức năng cụ thể và giao tiếp với nhau qua giao thức mạng hoặc cơ chế truyền tin.

Trong đồ án FailSafe, hệ thống được container hóa hoàn toàn bằng Docker, bao gồm các container chính:

- Web Service (Flask): Đảm nhận vai trò Frontend và API Gateway, chịu trách nhiệm nhận yêu cầu từ người dùng và hiển thị kết quả.
- Worker Service (Celery): Đảm nhận vai trò xử lý logic nghiệp vụ nặng (Deep Analysis), chạy ngầm dưới nền.
- Vector Database (ChromaDB): Dịch vụ lưu trữ và truy vấn dữ liệu vector độc lập.
- Message Broker (Redis): Cầu nối trung gian để điều phối dữ liệu giữa Web và Worker.

Lợi ích của kiến trúc này là khả năng cô lập lỗi (một dịch vụ lỗi không làm sập toàn bộ hệ thống) và khả năng mở rộng linh hoạt (có thể tăng số lượng Worker để xử lý nhiều yêu cầu hơn mà không cần sửa đổi mã nguồn Web).

2.5.2. Cơ chế Hàng đợi tác vụ (Task Queue) và Xử lý bất đồng bộ

Quy trình kiểm chứng thông tin (Fact-checking) là một tác vụ tốn kém tài nguyên và thời gian (Compute-intensive & Time-consuming). Một yêu cầu xử lý đầy đủ bao gồm nhiều bước: gọi LLM, cào dữ liệu web, phân tích văn phong... có thể mất từ vài chục giây đến vài phút.

Nếu xử lý đồng bộ (Synchronous) trên Web Server, người dùng sẽ phải chờ đợi rất lâu và server có thể bị quá tải (Time-out). Do đó, đề án áp dụng cơ chế Xử lý bất đồng bộ (Asynchronous Processing) sử dụng Celery và Redis.

Quy trình hoạt động:

1. Tiếp nhận: Khi người dùng gửi văn bản, Web Service không xử lý ngay mà đóng gói yêu cầu thành một "Message" và đẩy vào hàng đợi (Redis Queue).
2. Phản hồi nhanh: Web Service trả về ngay lập tức một Task ID cho người dùng, thông báo rằng yêu cầu đang được xử lý (trạng thái PENDING hoặc PROGRESS).
3. Xử lý ngầm: Worker Service (đang lắng nghe hàng đợi) nhận Message và bắt đầu thực hiện quy trình phân tích sâu. Trong quá trình chạy, Worker liên tục cập nhật trạng thái tiến độ (ví dụ: "Đang cào dữ liệu...", "Đang phân tích logic...") vào Redis.
4. Cập nhật: Giao diện người dùng (Frontend) định kỳ (Polling) hoặc sử dụng kết nối thời gian thực để lấy trạng thái từ Redis thông qua Task ID và hiển thị thanh tiến trình cho người dùng.

Cơ chế này đảm bảo trải nghiệm người dùng mượt mà (Non-blocking UI) và tối ưu hóa việc sử dụng tài nguyên hệ thống.

Chương 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu

3.1.1. Yêu cầu chức năng (Functional Requirements)

Hệ thống FailSafe được thiết kế để đáp ứng các nhu cầu cốt lõi của người dùng cuối (nhà báo, nhà nghiên cứu, người quản lý thông tin) thông qua các chức năng chính sau:

1. Tiếp nhận và Xử lý đầu vào đa dạng:
 - Hệ thống cho phép người dùng nhập liệu dưới dạng văn bản trực tiếp (Text) hoặc đường dẫn liên kết (URL).
 - Đối với đầu vào là URL, hệ thống phải có khả năng tự động cào (scrape), làm sạch nội dung và trích xuất văn bản chính để phân tích.
2. Sàng lọc và Cảnh báo sớm (Screening & Early Warning):
 - Tự động phân tích nguồn tin (dựa trên tên miền) để cảnh báo nếu nguồn tin có lịch sử không đáng tin cậy hoặc thiên kiến chính trị rõ rệt.
 - Phân tích phong cách viết (Stylometry) để phát hiện các dấu hiệu của tin giật gân, kích động cảm xúc (sensationalism).
 - Có khả năng "dừng sớm" (Early Exit) nếu phát hiện nội dung là rác hoặc quá kém chất lượng để tiết kiệm tài nguyên xử lý.
3. Kiểm chứng thông tin chuyên sâu (Deep Fact-checking):
 - Tự động phân rã văn bản thành các tuyên bố đơn lẻ (Atomic Claims).
 - Tìm kiếm bằng chứng từ hai nguồn: Cơ sở tri thức nội bộ (Knowledge Base) và Internet (Web Search).
 - Đánh giá độ chính xác của từng tuyên bố dựa trên bằng chứng thu thập được, phân loại theo các nhãn: Đúng (Supported), Sai (Refuted), hoặc Không liên quan/Không đủ dữ liệu (Irrelevant/Inconclusive).
4. Báo cáo và Giải thích trực quan (Reporting & Visualization):
 - Hiển thị kết quả dưới dạng Dashboard trực quan.
 - Cung cấp báo cáo tổng hợp (Executive Summary) do AI viết, đánh giá mức độ sự thật tổng thể (Factuality Score).
 - Hiển thị chi tiết bằng chứng cho từng tuyên bố, bao gồm trích dẫn nguồn (Source URL) và lý luận của AI (tại sao đúng/sai).
 - Trực quan hóa logic bài viết thông qua biểu đồ (Chart) và đồ thị mạng lưới (Argumentation Graph).
5. Quản lý tiến trình thời gian thực:
 - Hiển thị thanh tiến trình (Progress Bar) và trạng thái xử lý chi tiết từng bước (ví dụ: "Đang tìm kiếm bằng chứng...", "Đang phân tích logic...") để người dùng nắm bắt được hoạt động của hệ thống.

3.1.2. Yêu cầu phi chức năng (Non-functional Requirements)

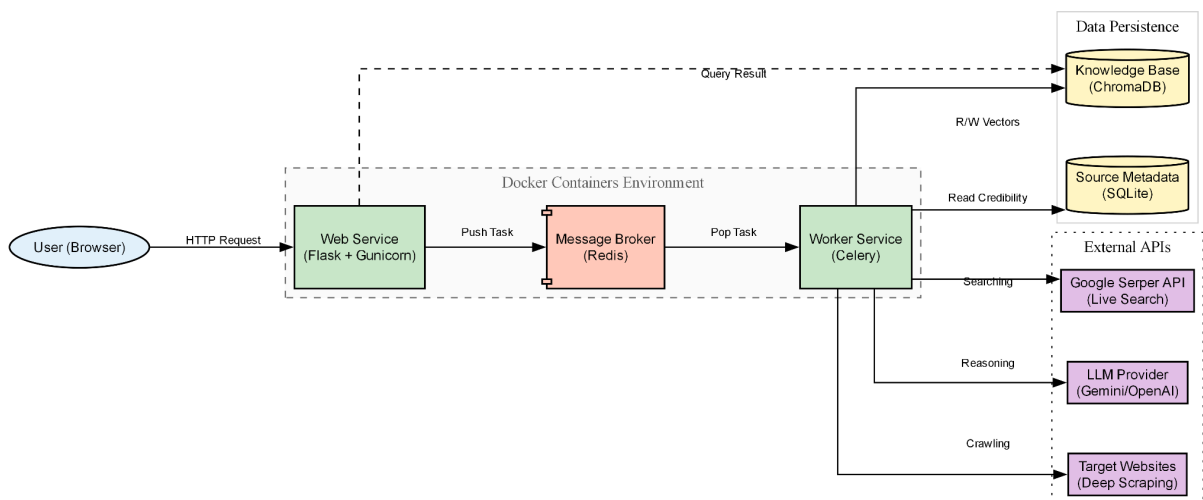
1. Hiệu năng và Độ trễ (Performance):
 - Hệ thống phải xử lý được các tác vụ nặng (Deep Analysis) mà không làm treo giao diện người dùng (Non-blocking UI).
 - Thời gian phản hồi cho bước sàng lọc nhanh (Screening) phải dưới 5 giây. Thời gian cho kiểm chứng sâu (Deep Check) chấp nhận được trong khoảng 30s - 120s tùy độ dài văn bản.
2. Khả năng mở rộng (Scalability):

- Hệ thống phải được thiết kế dạng module hóa (Microservices) để dễ dàng mở rộng số lượng Worker khi lượng người dùng tăng lên.
 - Cơ sở dữ liệu Vector phải có khả năng lưu trữ và truy vấn hiệu quả khi số lượng sự thật (Facts) tích lũy lớn dần theo thời gian.
3. Tính minh bạch (Transparency/Explainability):
- Hệ thống không được phép hoạt động như một "hộp đen". Mọi kết luận đưa ra bắt buộc phải đi kèm với trích dẫn nguồn gốc (Citation) và giải thích lý do (Reasoning).
4. Tính ổn định và Khả năng chịu lỗi (Reliability):
- Hệ thống phải có cơ chế tự động thử lại (Retry mechanism) khi gặp lỗi kết nối mạng hoặc lỗi API từ bên thứ ba (LLM, Search Engine).
 - Phải xử lý ngoại lệ tốt (Graceful degradation) để không bị sập (crash) khi gặp đầu vào không hợp lệ.

3.2. Kiến trúc hệ thống tổng thể

Dựa trên các yêu cầu phân tích, nhóm dự án thiết kế FailSafe theo kiến trúc Microservices (Vi dịch vụ), triển khai trên nền tảng Docker Container. Kiến trúc này giúp tách biệt các thành phần xử lý, đảm bảo tính độc lập, dễ dàng bảo trì và mở rộng. Hệ thống bao gồm 4 khối thành phần chính giao tiếp với nhau qua mạng nội bộ Docker:

3.2.1. Sơ đồ kiến trúc



Ảnh 3.1 Sơ đồ kiến trúc

- Client (User): Gửi Request HTTP -> Web Service.
- Web Service: Gửi Task => Redis (Message Broker) <= Nhận Task => Worker Service.
- Worker Service: Truy vấn/Lưu trữ => ChromaDB & SQLite.
- Web Service: Truy vấn kết quả => Redis & ChromaDB.

3.2.2. Mô tả các thành phần chi tiết

1. Web Service (Frontend & API Gateway)

- Công nghệ: Python Flask, HTML/JS/CSS (Bootstrap), Socket.IO (hoặc Polling).

- Vai trò:
 - + Cung cấp giao diện người dùng (GUI) để nhập liệu và xem báo cáo.
 - + Đóng vai trò là API Gateway tiếp nhận yêu cầu từ người dùng.
 - + Thực hiện các bước tiền xử lý nhẹ (như kiểm tra URL hợp lệ).
 - + Đẩy các tác vụ kiểm chứng nặng vào hàng đợi (Queue) và trả về mã theo dõi (Task ID) cho người dùng.
 - + Truy xuất kết quả từ Database để hiển thị khi tác vụ hoàn tất.

2. Message Broker (Trung gian truyền tin)

- Công nghệ: Redis (Remote Dictionary Server).
- Vai trò:
 - + Hoạt động như một "hàng đợi tin nhắn" (Message Queue) giữa Web Service và Worker Service.
 - + Lưu trữ trạng thái tạm thời của các tác vụ (Pending, Processing, Success, Failure) và thông tin tiến độ (Progress meta-data) để Web Service có thể cập nhật cho người dùng theo thời gian thực.
 - + Giúp điều tiết lưu lượng: Nếu lượng người dùng tăng đột biến, các yêu cầu sẽ được xếp hàng trong Redis chờ Worker xử lý dần, tránh làm sập hệ thống.

3. Worker Service (Xử lý nghiệp vụ lỗi)

- Công nghệ: Celery, PyTorch, LangChain, Sentence-Transformers.
- Vai trò:
 - + Đây là "bộ não" của hệ thống, chạy ngầm (Background Process).
 - + Lắng nghe hàng đợi Redis, khi có tác vụ mới sẽ tự động nhận về xử lý.
 - + Thực thi toàn bộ quy trình Fact-checking phức tạp: Gọi LLM (Gemini/GPT), chạy các mô hình AI cục bộ (Coreference, Stylometry), cào dữ liệu Web (Scraping).
 - + Ghi kết quả cuối cùng vào Database.

4. Data Persistence Layer (Tầng dữ liệu)

Hệ thống sử dụng mô hình lưu trữ lai (Hybrid Storage) để tối ưu hóa cho từng loại dữ liệu:

- SQLite (Relational DB): Lưu trữ dữ liệu có cấu trúc như thông tin các nguồn tin (Domain, Credibility Score, Bias Rating) phục vụ cho bước sàng lọc nhanh.
- ChromaDB (Vector Database): Lưu trữ dữ liệu phi cấu trúc dưới dạng Vector Embeddings.
 - + Collection Lưu trữ các tuyên bố đã được kiểm chứng (Knowledge Base).
 - + Collection Lưu trữ các mẫu văn bản để học máy (cho module Screening Advisor).

3.2.3. Cơ chế giao tiếp

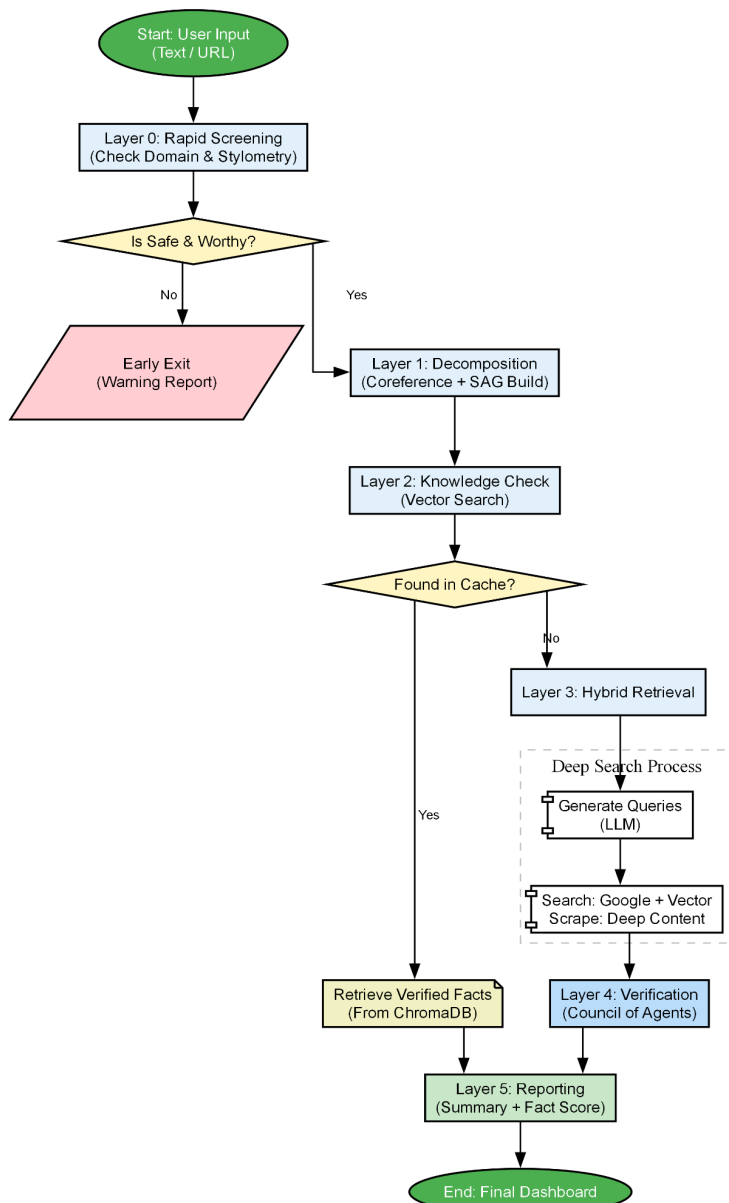
- Web => Redis: Đẩy Job (Producer).
- Worker => Redis: Lấy Job và cập nhật trạng thái (Consumer).
- Worker <=> External APIs: Gọi Google Serper API để tìm tin tức, gọi LLM APIs (OpenAI/Google) để suy luận.
- Worker <=> Databases: Đọc/Ghi dữ liệu nguồn tin và vector.

Kiến trúc này đảm bảo nguyên lý "Non-blocking": Web Service luôn phản hồi ngay lập tức cho người dùng, trong khi các tác vụ nặng được xử lý bất đồng bộ bởi Worker, mang lại trải nghiệm mượt mà và hiệu năng cao.

3.3. Thiết kế luồng xử lý dữ liệu (The Pipeline Design)

Quy trình kiểm chứng thông tin của FailSafe không phải là một bước đơn lẻ mà là một đường ống xử lý tuần tự (Sequential Pipeline) gồm 5 giai đoạn chính. Mỗi giai đoạn đóng vai trò như một bộ lọc, giúp tinh chỉnh dữ liệu từ dạng văn bản thô sơ thành tri thức đã được xác thực.

3.3.1. Sơ đồ luồng dữ liệu tổng quát



Ảnh 3.2 Sơ đồ luồng dữ liệu tổng quát

Input → [Layer 0] Screening → [Layer 1] Decomposition → [Layer 2] Caching Check → [Layer 3] Retrieval → [Layer 4] Verification → Output (Report)

3.3.2. Chi tiết từng giai đoạn xử lý

Giai đoạn 0: Sàng lọc nhanh (Rapid Screening Layer)

- Đầu vào: Văn bản thô hoặc URL.
- Hoạt động:
 - + Metadata Analysis: Trích xuất tên miền (Domain) từ URL, đối chiếu với cơ sở dữ liệu SQLite để xác định độ tin cậy (Trust Level) và thiên kiến chính trị (Bias).
 - + Stylometry Analysis: Phân tích các chỉ số văn phong (tỷ lệ từ ngữ giật gân, độ phức tạp entropy) để đánh giá tính chất "câu view" của bài viết.
- Đầu ra: Quyết định "Đi tiếp" hoặc "Dừng sớm" (Early Exit) kèm theo cảnh báo nếu phát hiện nguồn tin rác.

Giai đoạn 1: Phân rã và Cấu trúc hóa (Decomposition & Structuring)

- Đầu vào: Văn bản đã qua sàng lọc.
- Hoạt động:
 - + Coreference Resolution: Sử dụng mô hình FastCoref để thay thế các đại từ (he, she, it...) bằng danh từ cụ thể, giúp câu văn rõ nghĩa khi tách rời ngữ cảnh.
 - + SAG Construction: Sử dụng LLM để phân rã văn bản thành các tuyên bố nguyên tử (Atomic Claims) và xây dựng Đồ thị lập luận (SAG).
 - + Checkworthiness Detection: Lọc bỏ các câu là ý kiến cá nhân (Opinion) hoặc câu hỏi, chỉ giữ lại các tuyên bố mang tính sự thật (Factual Claims) cần kiểm chứng.
- Đầu ra: Danh sách các tuyên bố sạch (Check-worthy Claims).

Giai đoạn 2: Kiểm tra Bộ nhớ đệm (Caching Check)

- Đầu vào: Danh sách tuyên bố sạch.
- Hoạt động:
 - + Hệ thống chuyển đổi từng tuyên bố thành Vector Embedding.
 - + Truy vấn trong ChromaDB (Knowledge Base) để tìm các tuyên bố tương tự đã từng được kiểm chứng trước đó (Semantic Similarity Search).
- Đầu ra: Phân loại tuyên bố thành 2 nhóm: "Đã có kết quả" (lấy từ Cache) và "Cần kiểm tra mới" (New Claims).

Giai đoạn 3: Truy xuất bằng chứng (Retrieval Layer)

- Đầu vào: Các tuyên bố cần kiểm tra mới.
- Hoạt động:
 - + Query Generation: Tạo ra các từ khóa tìm kiếm tối ưu cho từng tuyên bố.
 - + Hybrid Search: Thực hiện song song tìm kiếm trên Web (Google Serper) và tìm kiếm Vector nội bộ.
 - + Deep Scraping: Tự động truy cập vào các URL uy tín tìm được, tải về nội dung đầy đủ để làm bằng chứng.
- Đầu ra: Tập hợp các đoạn văn bản bằng chứng (Evidences) liên quan cho từng tuyên bố.

Giai đoạn 4: Xác thực và Phản biện (Verification Layer - The Council)

- Đầu vào: Cặp (Tuyên bố, Bằng chứng).
- Hoạt động:
 - + Kích hoạt mô hình "Hội đồng thẩm định" (Council of Agents).
 - + Các tác nhân AI (Logician, Researcher, Skeptic) phân tích mối quan hệ giữa Tuyên bố và Bằng chứng (Hỗ trợ/Phản bác/Không liên quan) dựa trên logic và độ tin cậy nguồn tin.
 - + Tổng hợp phiếu bầu để đưa ra kết luận cuối cùng (Verdict).
- Đầu ra: Kết quả xác thực chi tiết (Label + Reasoning).

Giai đoạn 5: Tổng hợp và Báo cáo (Reporting Layer)

- Đầu vào: Kết quả xác thực của tất cả các tuyên bố.
- Hoạt động:
 - + Tính toán điểm số sự thật tổng thể (Factuality Score).
 - + LLM đóng vai trò "Tổng biên tập" (Editor-in-Chief) viết một báo cáo tóm tắt (Executive Summary).
 - + Lưu các kết quả mới vào Knowledge Base để học tập.
- Đầu ra: Báo cáo hoàn chỉnh gửi về cho người dùng.

3.4. Thiết kế Chi tiết các Module chính

3.4.1. Module Sàng lọc (Screening Module)

Module này chịu trách nhiệm đánh giá sơ bộ chất lượng đầu vào, đóng vai trò "người gác cổng" đầu tiên.

A. Class

- Chức năng: Phân tích độ tin cậy của tên miền (Domain Reputation).
- Cơ chế:
 - Trích xuất tên miền từ URL.
 - Tra cứu trong bảng sources của SQLite.
 - Nếu không tìm thấy trong DB, sử dụng LLM để đánh giá nóng (On-the-fly evaluation) về độ thiên kiến (Bias) và độ tin cậy (Credibility).

B. Class

- Chức năng: Phân tích định lượng phong cách viết.
- Thuật toán:
 - Tính độ hỗn loạn thông tin (Shannon Entropy) để phát hiện văn bản lặp lại, nghèo nàn.
 - Đếm từ khóa giật gân (Sensational words) dựa trên từ điển định sẵn.
 - Tính toán tỷ lệ chữ in hoa (Uppercase ratio) - dấu hiệu của việc "hét lớn" trong văn bản kém chất lượng.

3.4.2. Module Truy xuất (Retrieval Module)

Đây là module cầu nối giữa hệ thống và thế giới tri thức bên ngoài.

- Class Sử dụng LLM để biến đổi một câu trần thuật (Tuyên bố) thành các câu hỏi nghi vấn (Search Queries) tối ưu cho công cụ tìm kiếm.

B. Class

- a. Vector Search: Tìm kiếm trong ChromaDB với ngưỡng tương đồng (Threshold) được cấu hình chặt chẽ (ví dụ: 0.2) để đảm bảo chỉ lấy những kết quả thực sự liên quan.
- b. Web Search (Gọi API Google Serper để lấy danh sách URL. Sau đó kích hoạt DeepScraper sử dụng thư viện trafilatura để cào nội dung sạch (Text-only) từ các trang web uy tín, loại bỏ quảng cáo và mã HTML thừa.

3.4.3. Module Xác thực (Verification Module)

Module quan trọng nhất, nơi diễn ra quá trình suy luận.

A. Class

- a. Đầu vào: Một tuyên bố và danh sách bằng chứng.
- b. Cấu trúc Prompt: Sử dụng kỹ thuật "Nhập vai" (Persona Prompting) để khởi tạo 3 tác nhân ảo:
 - Logician: Kiểm tra lỗi logic (A $\rightarrow$ B có hợp lý không?).
 - Researcher: Kiểm tra độ mạnh yếu của bằng chứng (Evidence Hierarchy).
 - Skeptic: Kiểm tra tính giả khoa học (Pseudoscience check).
- c. Cơ chế bỏ phiếu: Kết quả cuối cùng được tổng hợp từ ý kiến của 3 tác nhân. Nếu có xung đột, hệ thống ưu tiên kết quả có độ tin cậy nguồn tin cao hơn.

3.5. Thiết kế Cơ sở dữ liệu

Hệ thống sử dụng mô hình dữ liệu lai (Hybrid Data Model) để tối ưu hóa cho từng loại truy vấn.

3.5.1. Cơ sở dữ liệu quan hệ (SQLite)

Dùng để lưu trữ dữ liệu có cấu trúc, ít thay đổi, phục vụ tra cứu nhanh (Lookup).

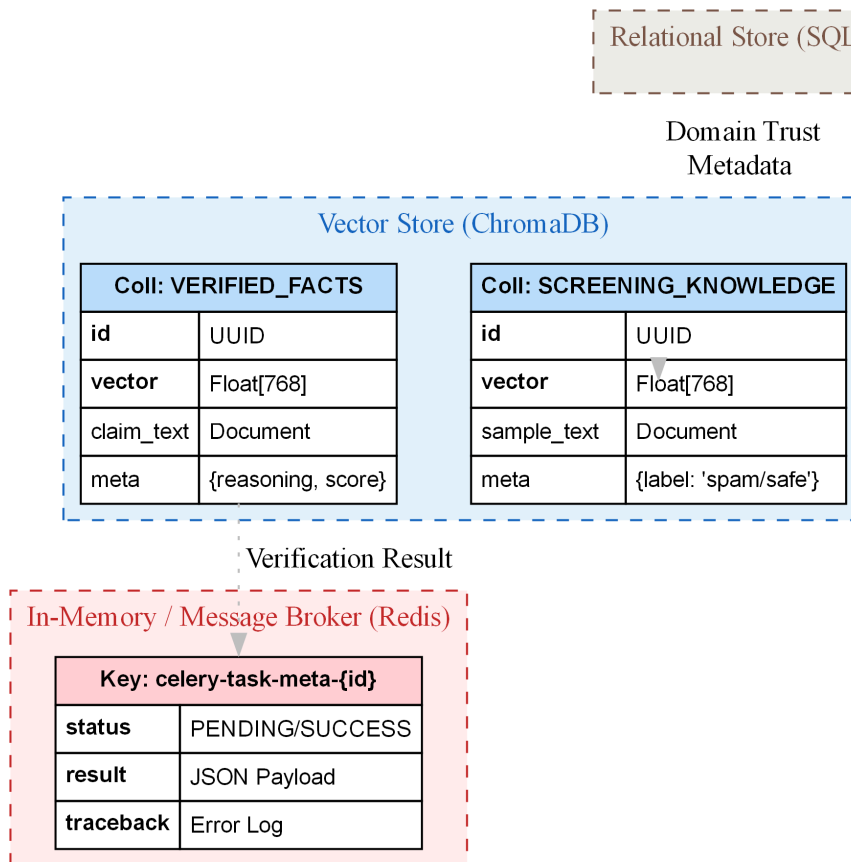
- Tên File: sources.db
- Bảng
 - + id (PK): Khóa chính.
 - + domain (Text, Unique): Tên miền (ví dụ: cnn.com).
 - + name (Text): Tên đầy đủ của nguồn tin.
 - + bias (Text): Đánh giá thiên kiến (Left, Center, Right).
 - + credibility (Text): Độ tin cậy (High, Mixed, Low).
 - + country (Text): Quốc gia xuất xứ.

3.5.2. Cơ sở dữ liệu Vector (ChromaDB)

Dùng để lưu trữ tri thức ngữ nghĩa, phục vụ tìm kiếm tương đồng (Semantic Search).

- Embedding Model: intfloat/e5-base-v2 (hoặc all-MiniLM-L6-v2).
- Collection
 - + Lưu trữ các tuyên bố đã được kiểm chứng.
 - + Metadata: Chứa điểm số sự thật (factuality), lý do kết luận (reasoning) và nguồn gốc.
 - + Mục đích: Hoạt động như bộ nhớ dài hạn (Long-term Memory) của hệ thống.
- Collection

- + Lưu trữ các mẫu văn bản đã phân loại (Tin rác, Tin ý kiến, Tin sự thật).
- + Mục đích: Giúp module ScreeningAdvisor học hỏi (Few-shot learning) để đưa ra lời khuyên nhanh cho các văn bản tương tự trong tương lai.



Ảnh 3.3 Sơ đồ Thiết kế Cơ sở dữ liệu

3.6. Thiết kế Giao diện người dùng (UI/UX)

Please enter the model's response:

Submit

© Skynet will take over the world

Ảnh 3.4 UI FailSafe

- Trang Nhập liệu (Thiết kế tối giản, tập trung vào ô nhập văn bản lớn. Hỗ trợ tự động nhận diện URL.

- Trang Chờ (Hiển thị trạng thái thời gian thực. Sử dụng WebSocket/Polling để cập nhật từng bước xử lý ("Đang tìm kiếm...", "Đang phân tích..."), giảm cảm giác chờ đợi cho người dùng.
- Trang Kết quả (Dashboard thông tin gồm:
 - + Điểm số tổng quan: Đồng hồ đo độ sự thật (Gauge Chart).
 - + Đồ thị lập luận: Hiển thị mối quan hệ giữa các ý bằng thư viện vis-network.
 - + Chi tiết: Danh sách các tuyên bố, bấm vào từng tuyên bố sẽ hiện ra các bằng chứng (thẻ bài) chứng minh hoặc bác bỏ.

Chương 4: CÀI ĐẶT VÀ ĐÁNH GIÁ THỰC NGHIỆM

4.1. Môi trường cài đặt và Công nghệ sử dụng

Để đảm bảo tính nhất quán và khả năng tái lập của hệ thống, dự án được phát triển và đóng gói hoàn toàn trên nền tảng Container hóa. Dưới đây là thông số kỹ thuật chi tiết của môi trường và các công nghệ lõi.

4.1.1. Môi trường phát triển và triển khai

- Hệ điều hành: Đa nền tảng (Windows 10/11, Ubuntu 20.04+, macOS). Hệ thống hoạt động trên nhân Linux bên trong Docker.
 - Ngôn ngữ lập trình: Python 3.11 (Phiên bản ổn định, tương thích tốt với các thư viện AI mới nhất như PyTorch và LangChain).
 - Nền tảng ảo hóa: Docker & Docker Compose.
- Mục đích: Đóng gói toàn bộ dependencies (thư viện, driver) vào các container độc lập, giúp việc triển khai (deployment) chỉ tốn một câu lệnh duy nhất, loại bỏ lỗi "works on my machine".

4.1.2. Các thư viện và Framework nòng cốt

Hệ thống được xây dựng dựa trên sự kết hợp của nhiều thư viện mã nguồn mở mạnh mẽ, được chia thành các nhóm chức năng sau:

A. Backend & Hệ thống phân tán:

- a. Flask (v3.x): Micro-framework nhẹ nhàng dùng để xây dựng Web Server và API Gateway.
- b. Celery (v5.x): Thư viện quản lý hàng đợi tác vụ phân tán (Distributed Task Queue), cho phép xử lý các tác vụ AI nặng dưới nền (background) một cách bất đồng bộ.
- c. Redis (v7.x): Đóng vai trò là Message Broker (trung chuyển tin nhắn giữa Web và Worker) và Result Backend (lưu trữ tạm thời kết quả xử lý).

B. Trí tuệ nhân tạo & Xử lý ngôn ngữ tự nhiên (AI/NLP):

- a. LangChain: Framework giúp kết nối và điều phối các mô hình ngôn ngữ lớn (LLMs) vào quy trình xử lý nghiệp vụ.
- b. Sentence-Transformers: Sử dụng để tạo ra các Vector Embeddings chất lượng cao (Model: all-MiniLM-L6-v2 hoặc intfloat/e5-base), phục vụ cho việc tìm kiếm ngữ nghĩa.
- c. FastCoref: Thư viện chuyên dụng giải quyết bài toán đồng tham chiếu (Coreference Resolution), giúp làm rõ nghĩa các đại từ trong văn bản.
- d. PyTorch (CPU/CUDA): Nền tảng tính toán tensor, hỗ trợ chạy các mô hình Deep Learning cục bộ.

C. Dữ liệu & Tìm kiếm:

- a. ChromaDB: Cơ sở dữ liệu Vector (Vector Database) mã nguồn mở, tối ưu hóa cho việc lưu trữ và truy vấn embeddings tốc độ cao.
- b. Google Generative AI SDK: Thư viện kết nối với mô hình Gemini Pro để thực hiện các tác vụ suy luận (Reasoning).
- c. Playwright & Trafilatura: Bộ công cụ mạnh mẽ dùng để cào dữ liệu (Scraping) từ các trang web động (Dynamic Web) và trích xuất nội dung văn bản sạch.

- d. BeautifulSoup4 (bs4): Phân tích cú pháp HTML để bóc tách dữ liệu từ các trang web tĩnh.

4.1.3. Cấu hình phần cứng khuyến nghị

Do hệ thống sử dụng các mô hình AI cục bộ (Local Models) cho các tác vụ như Coreference và Embedding, cấu hình phần cứng tối thiểu để hệ thống vận hành mượt mà là:

- CPU: Tối thiểu 4 cores (Khuyến nghị: 6 cores trở lên).
- RAM: Tối thiểu 8GB (Khuyến nghị: 16GB để chạy mượt Docker và load các Model vào bộ nhớ).
- Disk: SSD với dung lượng trống tối thiểu 10GB (để lưu Docker Images và Vector DB).
- Network: Kết nối Internet ổn định (để gọi External APIs như Google Serper, Gemini).

4.2. Triển khai hệ thống (Deployment)

Quy trình triển khai hệ thống FailSafe được tự động hóa tối đa nhờ Docker Compose. Người quản trị hệ thống chỉ cần thực hiện các bước sau để khởi chạy toàn bộ kiến trúc Microservices.

4.2.1. Cấu trúc thư mục dự án

Trước khi triển khai, mã nguồn dự án được tổ chức theo cấu trúc tiêu chuẩn:

```
fail-safe-system
├── factcheck/    # Mã nguồn lõi (Core Logic: Screening, Decompose, Verify...)
├── scripts/      # Các kịch bản khởi tạo dữ liệu (Setup scripts)
├── templates/    # Giao diện người dùng (HTML)
├── static/       # Tài nguyên tĩnh (CSS, JS, Images)
├── docker-compose.yaml # File cấu hình định nghĩa các dịch vụ (Services)
├── Dockerfile    # File cấu hình môi trường Python
├── requirements.txt # Danh sách thư viện phụ thuộc
├── tasks.py      # Định nghĩa các tác vụ Celery Worker
└── webapp.py     # Điểm khởi chạy Web Server (Flask App)
```

4.2.2. Quy trình khởi chạy

Bước 1: Thiết lập biến môi trường

Hệ thống yêu cầu các khóa API (API Keys) để kết nối với các dịch vụ bên ngoài.

Người dùng cần tạo file .env tại thư mục gốc và cung cấp các thông số:

```
GEMINI_API_KEY=...    # Khóa API cho mô hình suy luận
SERPER_API_KEY=...    # Khóa API cho tìm kiếm Google
OPENAI_API_KEY=...    # (Tùy chọn) Nếu sử dụng GPT-4
```

Bước 2: Xây dựng và Khởi động Container

Sử dụng Docker Compose để build images và kích hoạt các container. Quá trình này sẽ tự động tải các mô hình AI cần thiết (NLTK data, Sentence-Transformers) về máy.

```
# Build lại toàn bộ images để đảm bảo cập nhật code mới nhất
docker compose build --no-cache
```

```
# Khởi chạy hệ thống ở chế độ nền (Detached mode)
```

```
docker compose up -d
```

Sau khi lệnh hoàn tất, 4 container sẽ hoạt động đồng thời:

1. failsafe_redis: Lắng nghe tại cổng 6379.
2. failsafe_chroma: Lắng nghe tại cổng 8000.
3. failsafe_worker: Bắt đầu nhận task từ Redis.
4. failsafe_web: Phục vụ giao diện tại <http://localhost:2024>.

Bước 3: Khởi tạo dữ liệu ban đầu (Data Seeding)

Đối với lần chạy đầu tiên, hệ thống cần nạp dữ liệu nền tảng vào Cơ sở dữ liệu. Các lệnh sau được thực thi bên trong container worker:

```
# 1. Xây dựng dữ liệu thống kê văn phong (cho Stylometry Analyzer)
docker exec failsafe_worker python scripts/build_stylometry_corpus.py
```

```
# 2. Nhập dữ liệu độ tin cậy nguồn tin (MBFC) vào SQLite
docker exec failsafe_worker python scripts/import_mbfc_data.py
```

```
# 3. (Tùy chọn) Xây dựng Vector DB mẫu từ Wikipedia
docker exec failsafe_worker python scripts/build_vectordb.py
```

4.2.3. Kiểm tra trạng thái vận hành

Người quản trị có thể giám sát hoạt động của hệ thống thông qua logs thời gian thực:

```
# Xem logs của toàn bộ hệ thống
docker compose logs -f
```

```
# Hoặc xem logs riêng của Worker để debug quy trình suy luận
docker compose logs -f worker
```

Khi hệ thống báo "Celery Worker is initializing FactCheck instance...", nghĩa là quá trình khởi động đã thành công và sẵn sàng tiếp nhận yêu cầu từ người dùng.

4.3. Giao diện người dùng và Chức năng chi tiết

Giao diện người dùng (User Interface) của FailSafe được thiết kế theo phong cách tối giản (Minimalism), tập trung tối đa vào trải nghiệm đọc và tương tác với dữ liệu.

4.3.1. Trang Nhập liệu (Input Dashboard)

Đây là điểm tiếp xúc đầu tiên của người dùng với hệ thống.

1. Chức năng:

- a. Ô nhập liệu đa năng: Cho phép người dùng dán (paste) một đoạn văn bản cần kiểm tra hoặc dán đường link (URL) của một bài báo trực tuyến.
 - b. Tự động nhận diện: Hệ thống tự động phát hiện nếu đầu vào là URL để kích hoạt chế độ cào dữ liệu (Web Scraping).
 - c. Submit: Nút bấm gửi yêu cầu xử lý lên Server.
2. Giao diện: Thiết kế sạch sẽ với logo FailSafe, ô textarea lớn ở trung tâm và nút Submit nổi bật.

4.3.2. Trang Theo dõi tiến độ (Live Progress Tracking)

Do quá trình kiểm chứng chuyên sâu tốn nhiều thời gian (Deep Analysis), hệ thống cung cấp một trang chờ thông minh để giữ chân người dùng.

1. Chức năng:
 - a. Thanh tiến trình (Progress Bar): Hiển thị phần trăm hoàn thành công việc.
 - b. Nhật ký hoạt động (Live Logs): Hiển thị các thông báo trạng thái theo thời gian thực (ví dụ: "Step 1: Screening for misinformation...", "Step 2: Decomposing claims..."), giúp người dùng hiểu hệ thống đang làm gì ("Glass-box" approach).
 - c. Preview kết quả sớm: Ngay khi hệ thống phân rã xong các tuyên bố (Claims), danh sách tuyên bố sẽ hiện ra trước để người dùng xem trong lúc chờ xác thực.
2. Công nghệ: Sử dụng cơ chế Polling (gửi request định kỳ đến API /status/<task_id>) để cập nhật dữ liệu từ Redis mà không cần tải lại trang.

4.3.3. Trang Kết quả (Result Dashboard)

Đây là màn hình quan trọng nhất, hiển thị báo cáo toàn diện sau khi quá trình kiểm chứng hoàn tất.

1. Khu vực 1: Báo cáo Tổng quan (Executive Summary)
 - a. Hiển thị Điểm số sự thật (Factuality Score) dưới dạng phần trăm và nhãn (ví dụ: 85% - Highly Factual).
 - b. Tóm tắt số lượng tuyên bố: Tổng số, số câu Đúng (Supported), số câu Sai (Refuted).
 - c. Báo cáo tổng hợp: Một đoạn văn bản do AI viết, tóm tắt lại các vấn đề chính của bài viết, chỉ ra các lỗi ngụy biện hoặc sai lệch nghiêm trọng.
2. Khu vực 2: Văn bản gốc & Tương tác (Interactive Text)
 - a. Hiển thị toàn bộ văn bản gốc của người dùng.
 - b. Các câu (Tuyên bố) được tô màu nền (Highlight) tương ứng với độ chính xác: Màu xanh (Đúng), Màu đỏ (Sai), Màu vàng (Tranh cãi). Người dùng có thể click vào từng câu để xem chi tiết.
3. Khu vực 3: Chi tiết Bằng chứng (Evidence Detail)
 - a. Khi chọn một tuyên bố, hệ thống hiển thị danh sách các bằng chứng thu thập được trong các tab: Refutes (Phản bác), Supports (Hỗ trợ), Irrelevant (Không liên quan).
 - b. Mỗi bằng chứng đi kèm với: Trích dẫn nguồn (Source URL), Nội dung trích xuất và Lý luận của AI (Reasoning).
4. Khu vực 4: Đồ thị Lập luận (Argumentation Graph)

- a. Sử dụng thư viện vis-network để vẽ một đồ thị tương tác. Người dùng có thể kéo thả các nút, zoom in/out để khám phá mối quan hệ logic giữa các luận điểm trong bài viết.

4.4. Kịch bản kiểm thử và Đánh giá (Test Scenarios)

Để đánh giá hiệu quả hoạt động của hệ thống, nhóm nghiên cứu đã tiến hành thử nghiệm trên các mẫu dữ liệu đại diện cho 3 trường hợp phổ biến: Tin giả trắng trợn (Fake News), Tin tức khoa học chính xác (Factual News), và Tin tức chứa lỗi ngụy biện logic (Logical Fallacy).

4.4.1. Kịch bản 1: Phát hiện Tin giả & Giả khoa học (Pseudoscience)

1. Đầu vào: Một đoạn văn bản lan truyền thuyết âm mưu về việc Kim tự tháp Ai Cập do người ngoài hành tinh xây dựng.
"The Great Pyramid of Giza was built by aliens using anti-gravity technology. The stones are too heavy for humans to lift, and the coordinates of the pyramid exactly match the speed of light."
2. Kết quả xử lý của hệ thống:
 - a. Sàng lọc (Screening): Cảnh báo văn phong giật gân (Sensationalism Score cao).
 - b. Phân rã (Decomposition): Tách thành 2 tuyên bố: (1) Kim tự tháp do người ngoài hành tinh xây bằng công nghệ phản trọng lực; (2) Tọa độ khớp với tốc độ ánh sáng.
 - c. Xác thực (Verification):
 - Tuyên bố (1) bị đánh dấu REFUTES. Agent "The Skeptic" bác bỏ vì thiếu bằng chứng vật lý (Tier 1 Evidence) và áp dụng nguyên tắc Occam's Razor (giả thuyết con người dùng ròng rọc/đòn bẩy hợp lý hơn).
 - Tuyên bố (2) bị đánh dấu REFUTES hoặc IRRELEVANT. Agent "The Logician" chỉ ra lỗi ngụy biện (Coincidence/Cherry-picking) và lỗi niên đại (người Ai Cập cổ đại chưa biết đơn vị mét/giây để đo tốc độ ánh sáng).
 - d. Kết luận tổng thể: Factuality Score: 17% (Mostly False).

4.4.2. Kịch bản 2: Xác thực Tin tức Khoa học (Factual Verification)

1. Đầu vào: Một đoạn văn bản về biến đổi khí hậu.
"Global warming is primarily caused by human activities, specifically the emission of greenhouse gases like carbon dioxide. NASA data shows that the global average temperature has risen by about 1 degree Celsius since the late 19th century."
2. Kết quả xử lý của hệ thống:
 - a. Truy xuất (Retrieval): Tìm thấy nhiều nguồn uy tín (NASA.gov, IPCC reports) thông qua Google Serper.
 - b. Xác thực (Verification):
 - Tuyên bố về nguyên nhân con người: SUPPORTS (Đồng thuận khoa học cao).
 - Tuyên bố về nhiệt độ tăng 1 độ C: SUPPORTS (Khớp dữ liệu thống kê).

c. Kết luận tổng thể: Factuality Score: 100% (Highly Factual).

4.4.3. Kịch bản 3: Phát hiện Nguy hiểm Logic (Logical Fallacy Detection)

1. Đầu vào: Một đoạn văn bản chứa nguy hiểm tấn công cá nhân (Ad Hominem).
"Doctor Smith's research on vaccines is wrong because he was once fined for a speeding ticket. Therefore, vaccines are dangerous."
2. Kết quả xử lý của hệ thống:
 - a. Phân rã: Tách thành tuyên bố nhân quả: "Vaccine nguy hiểm VÌ Dr. Smith bị phạt tốc độ".
 - b. Xác thực (Verification):
 - Agent "The Logician" phát hiện lỗi Non Sequitur (Kết luận không suy ra từ tiền đề) và Ad Hominem (Tấn công tư cách cá nhân thay vì luận điểm khoa học).
 - Hệ thống kết luận mối quan hệ nhân quả này là sai lệch.
 - c. Kết luận tổng thể: Factuality Score: Low (Misleading). Báo cáo chỉ rõ lỗi logic trong lập luận của văn bản.

4.4.4. Đánh giá hiệu năng sơ bộ

Qua quá trình chạy thử nghiệm trên tập dữ liệu mẫu gồm 50 văn bản (độ dài trung bình 100 từ), hệ thống ghi nhận các chỉ số hiệu năng như sau:

Chỉ số	Giá trị trung bình	Ghi chú
Thời gian Sàng lọc (Screening)	~5 giây	Rất nhanh nhờ tra cứu SQLite cục bộ.
Thời gian Phân rã (Decomposition)	~10 giây	Phụ thuộc vào tốc độ phản hồi của LLM.
Thời gian Truy xuất (Retrieval)	~30 giây	Tốn thời gian nhất do độ trễ mạng khi cào dữ liệu Web.
Thời gian Xác thực (Verification)	~20 giây	Xử lý song song (Parallel) nhiều Claims cùng lúc.
Tổng thời gian xử lý	~50 - 70 giây	Chấp nhận được cho phân tích chuyên sâu (Deep Analysis).

Nhận xét: Hệ thống hoạt động ổn định, không xảy ra lỗi treo (crash). Thời gian xử lý tuy chưa đạt mức thời gian thực (Real-time) tức thì nhưng phù hợp với nhu cầu kiểm tra kỹ lưỡng của người dùng chuyên nghiệp.

4.5. Đánh giá và Thảo luận

Dựa trên kết quả thực nghiệm và các kịch bản kiểm thử đã thực hiện, nhóm nghiên cứu đưa ra những đánh giá khách quan về ưu điểm và hạn chế của hệ thống FailSafe.

4.5.1. Ưu điểm nổi bật

1. Khả năng giải thích vượt trội (Explainability):
Khác với các công cụ kiểm tra tin giả truyền thống (thường hoạt động như một "hộp đen"), FailSafe cung cấp một quy trình minh bạch tuyệt đối. Mỗi kết luận ĐÚNG/SAI đều đi kèm với một chuỗi lập luận logic (Chain-of-Thought) và trích dẫn nguồn gốc cụ thể. Điều này giúp người dùng không chỉ biết kết quả mà còn hiểu tại sao lại có kết quả đó.
2. Cơ chế chống ảo giác hiệu quả:
Việc áp dụng kiến trúc RAG kết hợp với "Hội đồng thẩm định đa tác nhân" đã giảm thiểu đáng kể tỷ lệ ảo giác của AI. Hệ thống thể hiện sự thận trọng cần thiết: khi không tìm thấy đủ bằng chứng, nó thà đưa ra kết luận "Inconclusive" (Chưa đủ kết luận) còn hơn là bịa đặt ra một sự thật sai lệch.
3. Kiến trúc linh hoạt và dễ mở rộng:
Thiết kế Microservices trên nền tảng Docker cho phép hệ thống dễ dàng mở rộng. Khi nhu cầu sử dụng tăng cao, có thể dễ dàng tăng số lượng container worker để xử lý song song nhiều yêu cầu hơn mà không ảnh hưởng đến kiến trúc tổng thể.

4.5.2. Hạn chế còn tồn tại

1. Độ trễ xử lý (Latency):
Với tổng thời gian trung bình khoảng 50-70 giây cho một văn bản, hệ thống chưa phù hợp cho các ứng dụng yêu cầu phản hồi tức thì (như tích hợp vào khung chat thời gian thực). Nguyên nhân chủ yếu đến từ độ trễ mạng khi thực hiện cào dữ liệu (Web Scraping) và thời gian suy luận của LLM.
2. Phụ thuộc vào nguồn dữ liệu Internet:
Hệ thống hoạt động dựa trên bằng chứng tìm thấy trên Web. Đối với các sự kiện vừa mới xảy ra (Breaking News) chưa được báo chí cập nhật, hoặc các thông tin nội bộ không công khai, hệ thống sẽ gặp khó khăn trong việc xác thực do thiếu dữ liệu đối chiếu.
3. Rào cản ngôn ngữ:
Hiện tại, hệ thống hoạt động tốt nhất với tiếng Anh do các bộ dữ liệu huấn luyện (MBFC) và Prompts được tối ưu cho ngôn ngữ này. Khả năng xử lý tiếng Việt và các ngôn ngữ khác phụ thuộc hoàn toàn vào năng lực đa ngôn ngữ của model LLM nền tảng (Gemini/GPT), đôi khi vẫn gặp lỗi ngữ pháp hoặc hiểu sai ngữ cảnh văn hóa địa phương.

4.5.3. Kết luận thực nghiệm

Nhìn chung, hệ thống FailSafe đã đáp ứng tốt các mục tiêu nghiên cứu đặt ra ban đầu. Nó chứng minh được tính khả thi của việc ứng dụng AI tạo sinh vào bài toán Fact-checking chuyên sâu, mở ra hướng đi mới cho việc tự động hóa quy trình kiểm duyệt nội dung với độ tin cậy cao.

Chương 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận chung

Trong bối cảnh bùng nổ thông tin và sự trỗi dậy của AI tạo sinh, vấn đề tin giả và ảo giác máy tính đang trở thành thách thức lớn đối với xã hội. Đồ án FailSafe đã được nghiên cứu và phát triển như một lời giải công nghệ cho bài toán này, hướng tới việc xây dựng một công cụ kiểm chứng thông tin tự động, minh bạch và có chiều sâu.

Qua quá trình thực hiện, đồ án đã đạt được những kết quả quan trọng sau:

1. Xây dựng thành công kiến trúc RAG lai (Hybrid RAG): Kết hợp hài hòa giữa tri thức nội bộ (Vector Database) và tri thức thời gian thực (Web Search), giúp hệ thống vừa có khả năng học hỏi tích lũy, vừa đảm bảo tính cập nhật.
2. Hiện thực hóa mô hình "Hội đồng thẩm định AI": Áp dụng thành công kỹ thuật Multi-agent với các vai trò chuyên biệt (Logic học, Nghiên cứu, Phản biện), giúp nâng cao đáng kể độ chính xác và khả năng giải thích của hệ thống so với các mô hình đơn lẻ.
3. Hoàn thiện sản phẩm phần mềm: Hệ thống đã được đóng gói hoàn chỉnh dưới dạng Microservices (Docker), có giao diện người dùng thân thiện, trực quan và hoạt động ổn định trong các kịch bản thử nghiệm thực tế.

Mặc dù vẫn còn những hạn chế nhất định về tốc độ xử lý và khả năng đa ngôn ngữ, nhưng FailSafe đã chứng minh được tiềm năng to lớn trong việc hỗ trợ con người lọc bỏ nhiễu loạn thông tin, góp phần xây dựng một môi trường số lành mạnh hơn.

5.2. Hướng phát triển trong tương lai

Để nâng cao hơn nữa hiệu quả và tính ứng dụng của hệ thống, nhóm nghiên cứu đề xuất các hướng phát triển tiếp theo:

1. Tối ưu hóa tốc độ thời gian thực (Real-time Optimization):
 - Nghiên cứu áp dụng các mô hình LLM nhỏ gọn hơn (Small Language Models - SLM) chạy cục bộ để giảm độ trễ suy luận.
 - Cải tiến thuật toán cào dữ liệu (Scraping) bằng kỹ thuật cào song song (Parallel Scraping) mạnh mẽ hơn.
2. Mở rộng hỗ trợ Đa phương thức (Multimodal Fact-checking):
 - Tích hợp thêm các module xử lý hình ảnh và video (sử dụng GPT-4 Vision hoặc Gemini Pro Vision) để kiểm chứng không chỉ văn bản mà cả các nội dung đa phương tiện, phát hiện ảnh chế (Deepfake).
3. Phát triển Plugin trình duyệt (Browser Extension):
 - Đóng gói hệ thống thành một tiện ích mở rộng (Extension) cài đặt trên Chrome/Edge, cho phép người dùng kiểm tra nhanh độ tin cậy của bài báo ngay khi đang lướt web mà không cần copy-paste sang trang khác.
4. Cải thiện khả năng Đa ngôn ngữ (Multilingual Support):
 - Tinh chỉnh (Fine-tune) lại các câu lệnh (Prompts) và bổ sung nguồn dữ liệu đánh giá độ tin cậy cho tiếng Việt và các ngôn ngữ phổ biến khác, hướng tới phục vụ cộng đồng người dùng toàn cầu.

TÀI LIỆU THAM KHẢO

- [1] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention is all you need,” in Advances in Neural Information Processing Systems, vol. 30, 2017. (Bài báo gốc về Transformer - nền tảng của mọi LLM).
 - [2] P. Lewis, E. Perez, A. Piktus, et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474, 2020. (Bài báo gốc về kỹ thuật RAG).
 - [3] J. Wei, X. Wang, D. Schuurmans, et al., “Chain-of-thought prompting elicits reasoning in large language models,” in Advances in Neural Information Processing Systems, vol. 35, 2022. (Cơ sở lý thuyết cho module suy luận đa bước).
 - [4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, 2019. (Cơ sở lý thuyết cho thư viện sentence-transformers dùng để tạo vector).
 - [5] Z. Ji, N. Lee, R. Frieske, et al., “Survey of hallucination in natural language generation,” ACM Computing Surveys, vol. 55, no. 12, pp. 1–38, 2023. (Tài liệu về vấn đề ảo giác của AI).
 - [6] S. Cohen, “FastCoref: Fast and Accurate Coreference Resolution,” GitHub repository, 2023. [Online]. Available: <https://github.com/shon-otmazgin/fastcoref>. (Thư viện xử lý đồng tham chiếu bạn dùng trong code).
 - [7] G. Team, “Gemini: A family of highly capable multimodal models,” arXiv preprint arXiv:2312.11805, 2023. (Tài liệu về model Gemini Pro).
- Tài liệu kỹ thuật & Website
- [8] LangChain AI, “LangChain Documentation,” [Online]. Available: <https://python.langchain.com/>.
 - [9] ChromaDB, “The AI-native open-source embedding database,” [Online]. Available: <https://www.trychroma.com/>.
 - [10] Media Bias/Fact Check (MBFC), “Methodology for evaluating media bias and credibility,” [Online]. Available: <https://mediabiasfactcheck.com/>. (Nguồn dữ liệu cho file import_mbfc_data.py).
 - [11] Docker Inc., “Docker Documentation,” [Online]. Available: <https://docs.docker.com/>.
 - [12] Celery Project, “Celery: Distributed Task Queue,” [Online]. Available: <https://docs.celeryq.dev/>.
 - [13] Framework Loki: An Open-source Tool for Fact Verification: [Libr-AI/OpenFactVerification: Loki: Open-source solution designed to automate the process of verifying factuality](#) (Tham khảo)